HyperCLI

Final Project Documentation

Nursayat Bashan SE-2521

Introduction

HyperCLI is a simple Python CLI project.

The project was made for practicing Python, OOP, and file handling.

The program works in terminal and accepts commands from user.

Main functions:
- search files
- organize files
- remove empty folders
- show logs
- show history

Project Goal

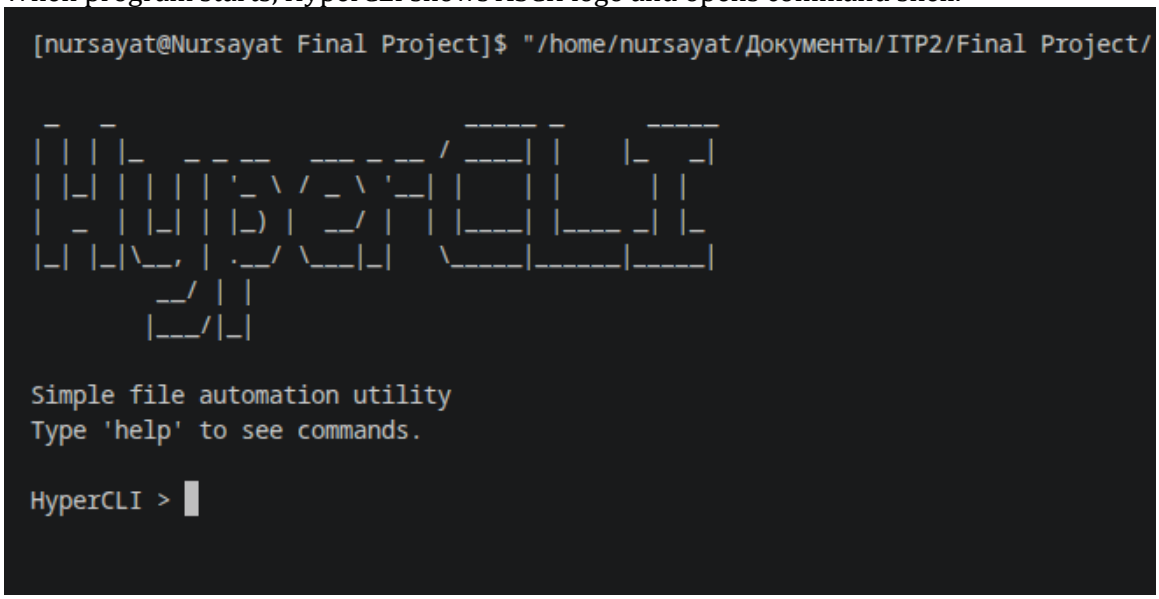
The goal of the project is to make a simple file automation utility.

The project also helped practice:
- OOP
- JSON
- generators
- decorators
- regex
- modular structure

Program Start

The project starts from main.py.

When program starts, HyperCLI shows ASCII logo and opens command shell.



Commands

The project supports commands:

- search
- organize
- cleanup
- logs
- history
- help
- exit

Example:

help

help organize

logs --page 2



Commands are handled by CommandProcessor.

Search System

The project contains file search system.

Example:

search --path ~/Downloads --ext .png

The program searches files using:
- os.walk()
- generators
- regex

Generator example:

yield os.path.join(root, file)

```
HyperCLI > search --path /home/nursayat/Загрузки/ --ext .docx

Search results:
+--------------------------+--------------------------+--------------------------+
| NAME                     | SIZE                     | DATE                     |
+--------------------------+--------------------------+--------------------------+
| .~perCLI_Documentation.d | 0.00MB                   | 2026-05-17               |
| HyperCLI_Documentation.d | 0.04MB                   | 2026-05-17               |
| .~2521_Sociology_Bilingu | 0.00MB                   | 2026-05-12               |
| Introduction to programm | 0.12MB                   | 2026-05-08               |
| .~troduction to programm | 0.00MB                   | 2026-05-14               |
| SE2521_Sociology_Bilingu | 0.02MB                   | 2026-05-12               |
| Assignment#4 (2).docx    | 2.97MB                   | 2026-05-11               |
+--------------------------+--------------------------+--------------------------+
```

File Organization

The project can organize files by extension.

Example:

organize --path ~/Downloads

The program:
- checks extension
- creates folders
- moves files

Rules are stored in rules.json.

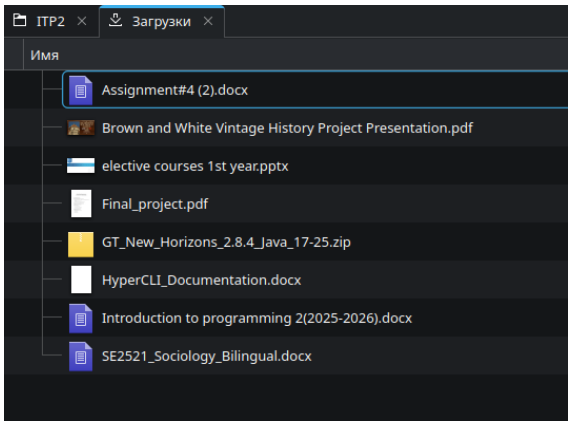
Example:

{ "extension": ".jpg", "target": "Images" }

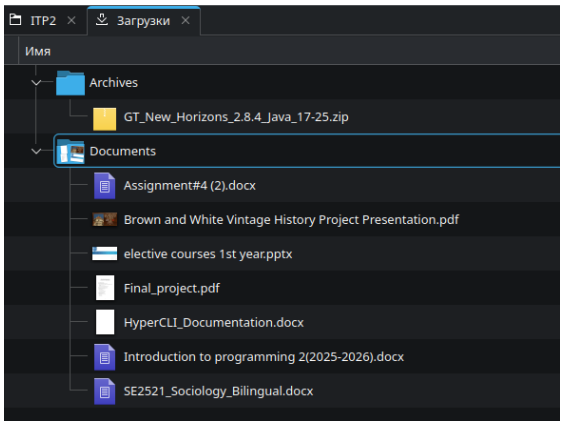
```
HyperCLI > organize --path /home/nursayat/Загрузки/

Organization complete. Files moved: 9. Failed: 0.
```

Before:



After:



Cleanup

The project can remove empty folders.

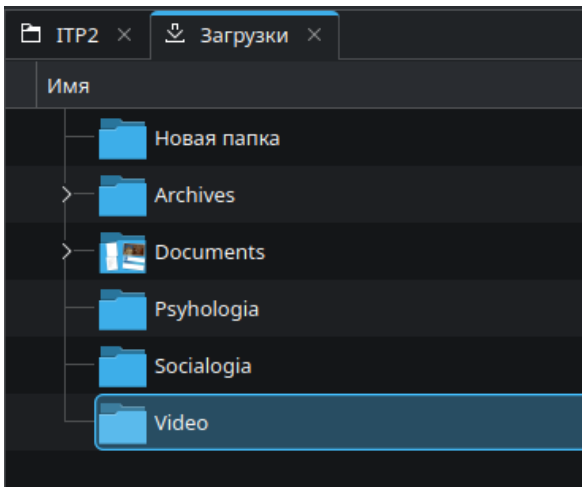
Example:

cleanup --path ~/Downloads

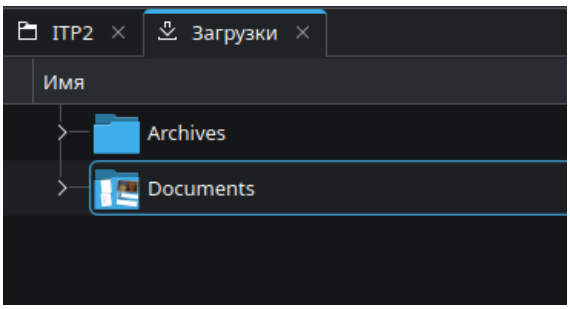
```
HyperCLI > cleanup --path /home/nursayat/Загрузки/

Cleanup complete. Empty folders removed: 4.
```

Before:



After:



Logs

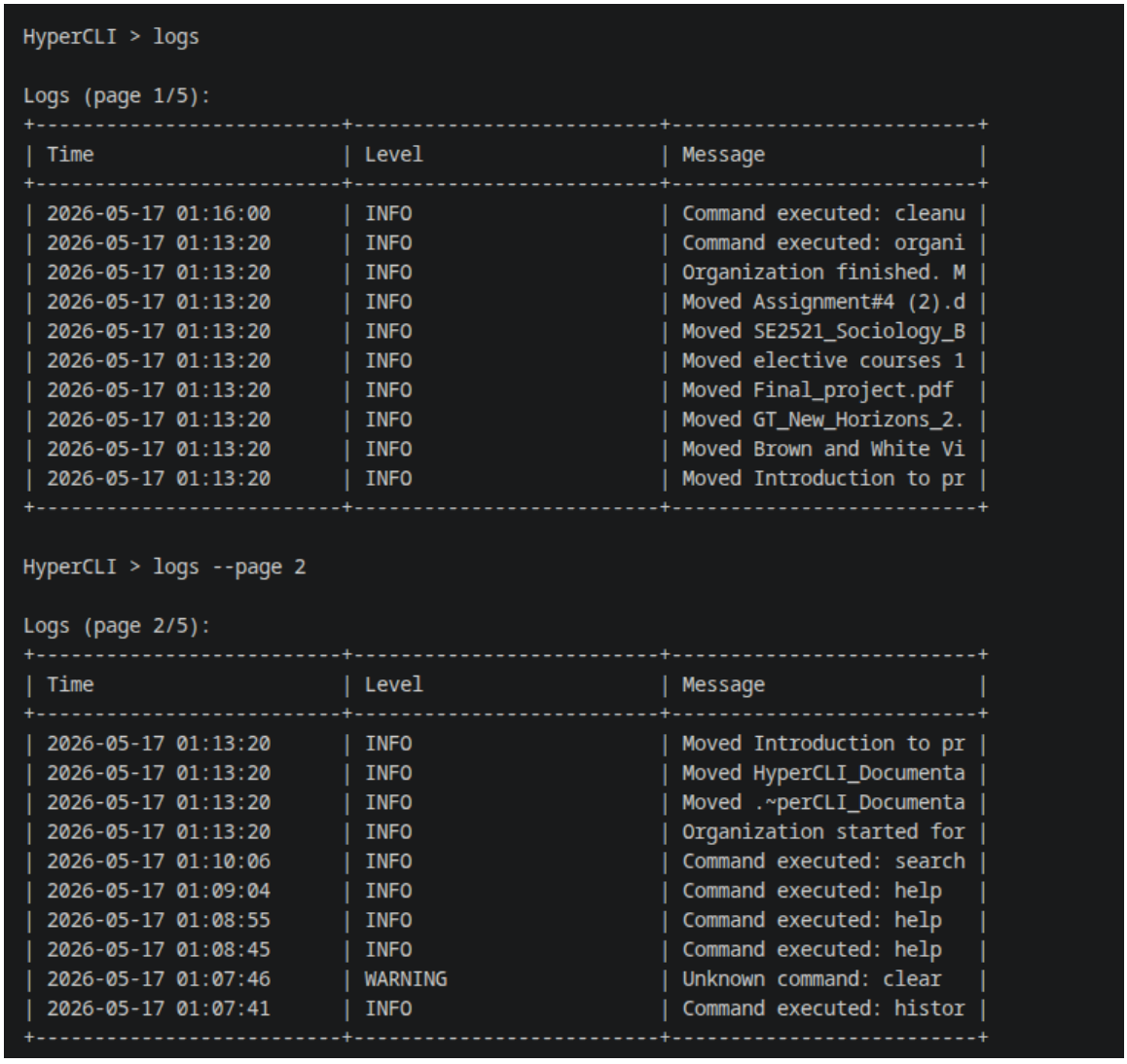
The project contains simple logging system.

Example:

logs
logs --page 2

Example log:

{ "level": "INFO", "message": "Moved photo.jpg to Images" }



History

The project stores command history in history.json.

Example:

history

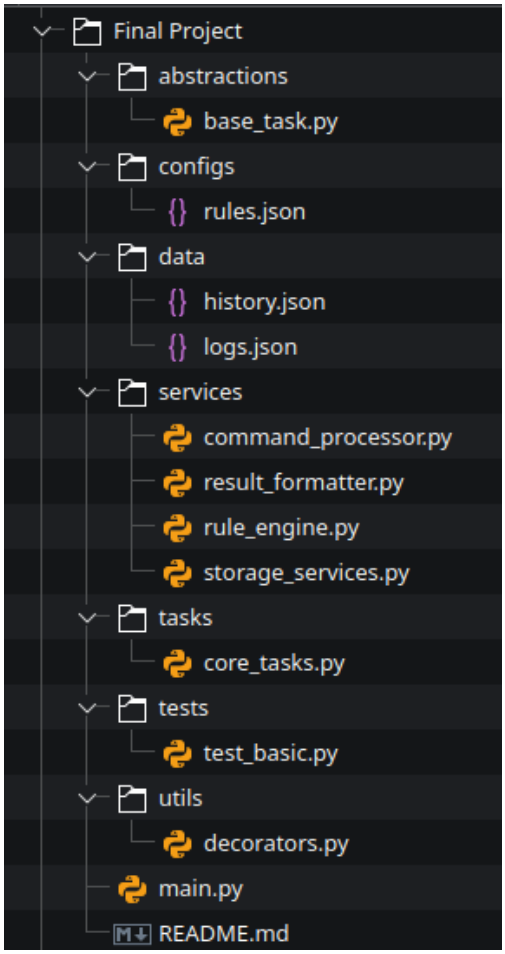
History stores:
- command
- status
- timestamp

```
HyperCLI > history

Recent actions:
+-------------------------+-------------------------+-------------------------+
| Time                    | Command                 | Status                  |
+-------------------------+-------------------------+-------------------------+
| 2026-05-12 08:54:39     | organize --path /home/us | SUCCESS                 |
| 2026-05-12 08:55:00     | logs                    | SUCCESS                 |
| 2026-05-14 09:11:12     | help                    | SUCCESS                 |
| 2026-05-14 09:11:58     | search --path /home/nurs | SUCCESS                 |
| 2026-05-14 09:12:16     | organize --path /home/nu | SUCCESS                 |
| 2026-05-14 09:12:30     | history                 | SUCCESS                 |
| 2026-05-14 09:12:36     | logs                    | SUCCESS                 |
| 2026-05-14 09:12:58     | help organize           | SUCCESS                 |
| 2026-05-14 09:13:13     | help search             | SUCCESS                 |
| 2026-05-14 09:13:23     | help logs               | SUCCESS                 |
| 2026-05-14 09:13:33     | help history            | SUCCESS                 |
| 2026-05-14 09:13:37     | help help               | SUCCESS                 |
| 2026-05-14 09:13:55     | logs                    | SUCCESS                 |
| 2026-05-14 09:14:03     | logs --page 2           | SUCCESS                 |
| 2026-05-14 09:14:13     | logs --page 3           | SUCCESS                 |
| 2026-05-14 09:48:53     | help                    | SUCCESS                 |
| 2026-05-14 09:50:50     | help                    | SUCCESS                 |
| 2026-05-17 01:07:36     | logs                    | SUCCESS                 |
| 2026-05-17 01:07:41     | history                 | SUCCESS                 |
| 2026-05-17 01:08:45     | help                    | SUCCESS                 |
| 2026-05-17 01:08:55     | help search             | SUCCESS                 |
| 2026-05-17 01:09:04     | help organize           | SUCCESS                 |
| 2026-05-17 01:10:06     | search --path /home/nurs | SUCCESS                 |
| 2026-05-17 01:13:20     | organize --path /home/nu | SUCCESS                 |
| 2026-05-17 01:16:00     | cleanup --path /home/nur | SUCCESS                 |
| 2026-05-17 01:16:42     | logs                    | SUCCESS                 |
| 2026-05-17 01:16:51     | logs --page 2           | SUCCESS                 |
+-------------------------+-------------------------+-------------------------+
```

Project Structure

The project is split into folders and modules.



OOP

The project uses OOP concepts.

Inheritance:
Tasks inherit from BaseTask.
Example:
class SearchTask(BaseTask):
Polymorphism:
Each task has its own execute() method.
Example:
task.execute(args)
Composition:
CommandProcessor contains task objects.

Advanced Python Features

The project also uses:
- generators
- decorators
- regular expressions

Decorator example:

@log_execution

Regex example:

re.search(r'--path\s+([^\s]+)', args)

Testing

The project contains basic tests in tests/test_basic.py.

The tests check:
- generators
- RuleEngine
- inheritance
- organize system

Example:

assert len(files) == 1

Conclusion

This project helped improve Python and OOP skills.

During development, the project used:
- file handling
- JSON
- generators
- decorators
- regex
- modular programming

The project also helped practice building CLI applications.